

Cloud Provider Analytics

Un pipeline **Lambda** para costos, soporte y consumo GenAI

Camila Lee (63382) Lucas Perri (62746) Prof. Diego Mosquera

43.200

eventos ingestados

5

zonas de data lake

7

tablas de serving

5

consultas de negocio

El problema

Un proveedor cloud responde tres preguntas, a dos velocidades.

TRES ÁREAS, TRES NECESIDADES

- **FinOps** — cuánto gasta cada cliente, en qué servicio, qué día
- **Soporte** — cuántos tickets críticos, dónde se rompió el SLA
- **Producto / GenAI** — cuántos tokens se consumen y cuánto cuestan

DOS VELOCIDADES

Near real-time

eventos de uso y costos incrementales

Batch

maestros de clientes, facturación mensual, encuestas

problema transversal · los datos llegan sucios y el esquema cambia a mitad del histórico.

Los datos reales

Antes de diseñar nada, medimos qué tan sucio estaba el dato.

LO QUE ENCONTRAMOS	CUÁNTO
value viene como número, string "100.0" o nulo	877 nulos
unit nulo teniendo value	2.038 filas
cost_usd_increment negativo	216 eventos
Evolución de esquema v1 → v2	corte 2025-07-18
carbon_kg y genai_tokens sólo en v2	32.400 / 43.200
Facturas con subtotal negativo	13 de 240
credits vacío	137 de 240
NPS fuera de rango	1 solo valor (101)

→ Cada decisión del pipeline sale de una de estas filas.

Propuesta de ingeniería

Cuatro decisiones que sostienen todo el diseño.

01

Data Lake por zonas

Landing → Bronze → Silver → Gold, más Quarantine.
Cada zona con una sola responsabilidad.

02

Patrón Lambda

Batch para estados, streaming para eventos. Cada dato se procesa según su naturaleza.

03

Calidad activa

3 reglas que aíslan lo inválido, y anomalías por consenso de 3 métodos estadísticos.

04

Serving query-first

Una tabla de Cassandra por consulta. La clave se diseña desde la pregunta.

Patrón Lambda

El dataset tiene dos naturalezas distintas.



POR QUÉ LAMBDA Y NO KAPPA

Los **CSV son estados** con cadencia humana: facturación mensual, altas de clientes.

El **JSONL son eventos** con cadencia de máquina.

Kappa exigiría inventar captura de cambios sobre CSV estáticos y sumar un broker replayable (Kafka), fuera del stack de la materia.

Zonas del lake

Cinco zonas, cinco responsabilidades.

ZONA	QUÉ HACE	QUÉ CONTIENE
Landing	Recibe	Archivos originales, inmutables
Bronze	Tipa	Parquet con esquema explícito, dedup, ingest_ts + source_file
Silver	Limpia	3 reglas, joins de enriquecimiento, flags de anomalía
Gold	Resume	5 marts pre-agregados, uno por consulta
Quarantine	Aísla	Filas inválidas, con la fila completa para reprocesar

→ Cualquier resultado raro se depura caminando las zonas hacia atrás.

Stack tecnológico

Qué usamos, y para qué.

Apache Spark motor distribuido; misma lógica en Colab o en cluster

Structured Streaming ingesta de los JSONL como tabla ilimitada

Parquet formato columnar particionado para las zonas del lake

Cassandra · AstraDB serving NoSQL, modelado por consulta

Google Colab entorno de ejecución reproducible

cassandra-driver carga distribuida desde los executors de Spark

Bronze

Mismo grano que la fuente, con tipos y procedencia.

ACCIONES

- Esquema explícito (StructType), nunca inferSchema
- Un solo esquema v1/v2, con carbon_kg y genai_tokens nullable
- value se lee como string y se castea con fallback
- Dedup por clave natural, ingest_ts + source_file en cada fila
- Streaming con watermark, dedup por event_id y checkpoint

RESULTADOS

DATASET	REGISTROS	PART. POR
customers_orgs	80	hq_region
users	800	role
billing_monthly	240	currency
support_tickets	1.000	category
usage_events	43.200	sink stream

El watermark

LO QUE CREÍAMOS

10 minutos alcanzaba, porque los archivos simulan micro-lotes de pocos minutos.

LO QUE MEDIMOS

Ninguno de los 120 archivos es un corte temporal. Cada uno cubre los 60 días completos.

WATERMARK	MICRO-LOTES	EVENTOS EN BRONZE
10 min	1 (todos juntos)	43.200 / 43.200
10 min	12 (maxFilesPerTrigger=10)	7.203 / 43.200
60 días	12	43.200 / 43.200

−83% Con 10 minutos se perdía el **83% de los eventos** apenas Spark partía la corrida en lotes. Sin error, sin warning: sólo menos filas en Gold.

Silver · calidad y quarantine

#	REGLA	POR QUÉ
1	event_id no nulo	sin clave no hay dedup ni idempotencia
2	cost_usd_increment $\geq$ -0,01	tolera redondeo, no costos negativos
3	unit no nulo si hay value	no hay consumo sin unidad de medida

40.956 filas válidas → Gold	2.244 en quarantine	43.200 suman el total
---	---	---

TRADE-OFF DOCUMENTADO · la regla 3 manda a quarantine filas con costo válido: **7.110,15 USD** y **139.949 tokens GenAI (5,5%)** que no llegan a Gold. Lo medimos en vez de ignorarlo.

Silver · anomalías

z-score

$$|x - \mu| / \sigma > 3$$

Se distorsiona con la misma cola que busca.

MAD

mediana de |desvíos|

Robusto, pero conservador.

p99

top 1%

Sin supuestos de distribución, pero bruto.

Anomalía sólo si coinciden **≥ 2 de 3** · estadísticos por servicio, porque los costos no están en la misma escala.

~~211~~ → **89**

anomaly_methods guarda qué métodos dispararon → va como `set<text>` en Cassandra. No recortamos el costo: un pico es justo lo que FinOps quiere ver.

El dato sucio

FX

El tipo de cambio de facturas en USD es ruido.

USD 0,855–1,118 (media 0,9977). Un USD≠1 no existe. EUR y ARS sí son plausibles.

decisión

forzar USD → 1,0

Global +0,07% (se cancela). Pero por (org, mes) el desvío mediano es **2,17%** y el máx **17,01%**.

CSAT

Parecía sucio y no lo estaba.

Distribución 0–7. La lectura fácil: "la escala es 1–5, el resto es ruido". Ajustada a una normal redondeada ($\mu=3,30$ $\sigma=1,26$) pega en todo el rango, colas incluidas.

decisión

no tocar nada

Una regla de rango habría mandado **40 filas legítimas** a quarantine.

NPS

Sólo un valor es inválido.

16 valores negativos parecen errores, pero son NPS válidos: la escala va de **−100** a **+100**.

decisión

un solo outlier real

El único fuera de rango es un **101**.

Gold

Cinco marts pre-agregados, uno por consulta.

MART	GRANO	FILAS	SIRVE
org_daily_usage_by_service	org × servicio × día	11.050	Q1 · Q2
tickets_by_org_date	org × día	944	Q3
revenue_by_org_month	org × mes	240	Q4
genai_tokens_by_org_date	org × día (genai)	1.131	Q5
cost_anomaly_mart	org × servicio × día	89	alerting FinOps

Features desde metric, no desde count(*): requests · cpu_hours · storage_gb_hours salen de sum(value WHERE metric = X).

Serving query-first

TABLA	PRIMARY KEY	SIRVE
org_daily_usage_by_service	((org_id, service), usage_date)	Q1
services_by_org	((org_id), service)	Q2 (índice)
tickets_by_org_date	((org_id), ticket_date)	Q3
revenue_by_org_month	((org_id), month)	Q4
genai_tokens_by_org_date	((org_id), usage_date)	Q5
cost_anomaly_mart	((org_id), usage_date, service)	alerting
org_daily_usage_stream	((org_id, service), usage_date)	speed layer

COLECCIONES CQL · donde el groupBy destruía info: anomaly_methods set<text> · severities map<text,int> · currencies set<text>

CARGA DISTRIBUIDA · foreachPartition: cada partición abre su sesión y manda INSERTs asíncronos. El mart nunca pasa por el driver.

Las 5 consultas

Q1 costos y requests diarios · org × servicio

\$340,53

pico del rango · 2025-08-24 · 390 requests (genai)

Q2 top-N servicios por costo · 14 días

genai	661,97
analytics	119,14
compute	112,08
database	67,41

Q3 tickets críticos y SLA breach · 30 días

fecha	tot	crít	breach
08-30	1	0	0%
08-23	1	1	0%
08-15	2	0	0%
08-08	1	1	0%

Q4 revenue mensual normalizado a USD

mes	net USD	cur
2025-08	1.426,11	USD
2025-07	3.155,10	EUR
2025-06	940,41	USD

Q5 tokens GenAI y costo por día

8.631

máximo diario · 2025-08-25 · \$92,05 estimado

NOTA · Q2

El top-N cruza particiones. Los servicios salen de services_by_org (una sola partición, sin scan) y el orden se hace del lado de la app.

Idempotencia

TRES MECANISMOS QUE SE APOYAN ENTRE SÍ

- 01 Archivos `replayables` · la fuente se puede volver a leer entera
- 02 Checkpoint · el streaming sabe por dónde iba
- 03 UPSERT por `clave` · lo ya escrito se pisa con el mismo valor

Si la corrida se cae a la mitad, el **checkpoint** sabe por dónde iba y lo ya escrito se **pisa con el mismo valor**.

TZ DE SESIÓN · fijada en UTC porque el dato es UTC — no la del SO.

LA PRUEBA: CONTAR · RECARGAR · CONTAR

TABLA	1ª	2ª	=
org_daily_usage_by_service	11.050	11.050	✓
tickets_by_org_date	944	944	✓
revenue_by_org_month	240	240	✓
genai_tokens_by_org_date	1.131	1.131	✓
cost_anomaly_mart	89	89	✓

6 tablas, seis conteos idénticos: recargar la corrida completa no cambia nada.

RANGOS DE FECHA · derivados de `max(usage_date)` del mart, no del reloj: da lo mismo hoy que en tres meses.

Qué nos llevamos

Tres lecciones del proyecto.

01 Un pipeline que anda no es lo mismo que un pipeline correcto: el watermark funcionaba por casualidad.

02 Un pipeline que da distinto en dos máquinas no está mal en una: está mal definido.

03 "Esto parece raro" no es lo mismo que "esto está mal": CSAT y NPS parecían sucios y no lo estaban.

Gracias.

Camila Lee · Lucas Perri
ITBA · Big Data 72.80 · 1C 2026